

Cloud-Based Data Analytics & AI Pipeline for Smart Healthcare

Technical Report — Cloud Computing & Big Data

1. Introduction & Objectives

This project implements an end-to-end, serverless data and analytics pipeline on AWS for a hospital consortium, building a proof of concept (POC) that turns raw operational data into actionable insights. The focus is data engineering and analytics rather than infrastructure: ingesting data into a data lake, cataloguing and transforming it, querying it with SQL, and presenting the results on an interactive dashboard.

The implemented scope (the **core path**) is **Amazon S3** (data lake) → **AWS Glue** (crawler, catalog, ETL) → **Amazon Athena** (SQL analytics) → **Amazon Quick Sight** (dashboard). All services run within the AWS Free Tier, and all data is synthetic — no real patient information is used, satisfying the privacy constraint.

The objectives addressed are: (1) design a serverless data pipeline; (2) implement ingestion, ETL and data cataloguing; (3) apply analytics to surface operational trends such as occupancy, wait times and patient readmission; and (4) visualise the outcomes on a dashboard.

2. Architecture

The architecture follows a classic data-lake pattern with **Bronze / Silver / Gold** zones inside a single S3 bucket. Data arrives two ways: batch CSV uploads and a real-time **Kinesis Data Firehose** stream. A **Step Functions** workflow automates the crawler and ETL job, and a single IAM role governs service access to S3 and Athena.

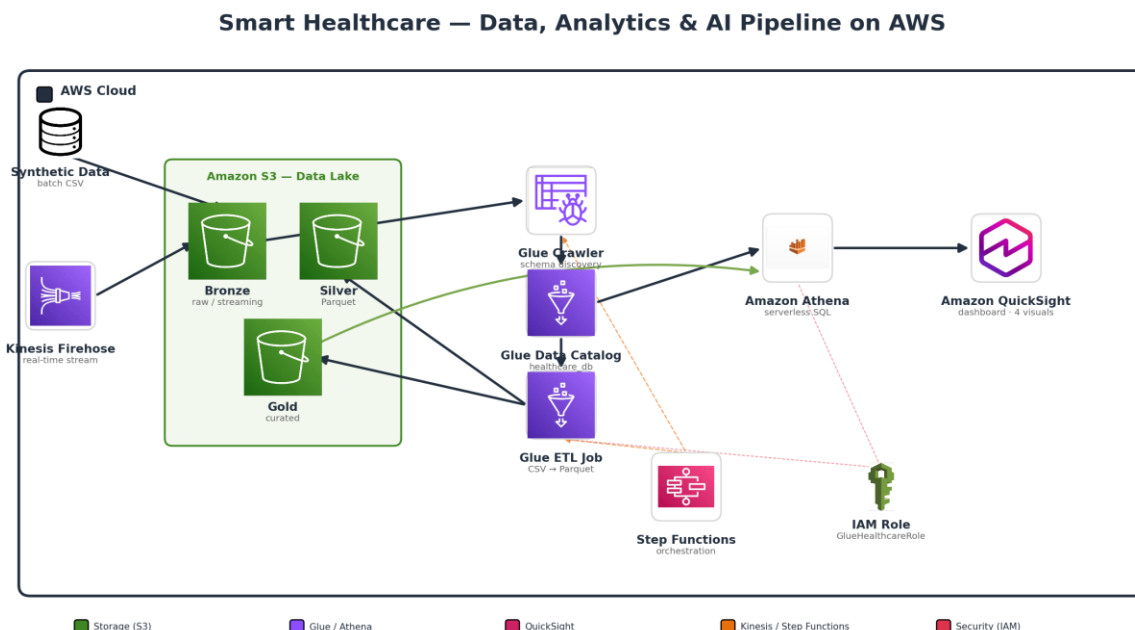


Figure 1 — End-to-end pipeline architecture (AWS Free Tier).

2.1 Component responsibilities

Layer	Service	Role in the pipeline
Ingestion	Kinesis Data Firehose + batch	Real-time stream + batch CSV into the Bronze zone
Storage	Amazon S3	Data lake with Bronze (raw CSV), Silver (cleaned Parquet) and Gold (curated) zones
Catalog	Glue Crawler + Data Catalog	Automatic schema discovery; central metadata store (healthcare_db)
ETL	AWS Glue Job	Transforms and types the data; writes Parquet to Silver/Gold
Query	Amazon Athena	Serverless SQL analytics over the catalogued tables
Visualization	Amazon QuickSight	Interactive dashboard with four operational visuals
Automation	AWS Step Functions	Orchestrates crawler then ETL job as one serverless workflow
Security	IAM (GlueHealthcareRole)	Grants Glue & QuickSight scoped access to S3 and Athena

3. Dataset

Three synthetic datasets were generated with a Python script (using NumPy/Pandas) and seeded for reproducibility. They contain deliberate, realistic correlations so that the analytics reveal meaningful patterns:

- **patients.csv** (5,000 rows) — one record per admission: demographics, department, admission type, diagnosis, length of stay and a 30-day readmission flag.
- **vitals.csv** (~50,000 rows) — IoT patient-monitoring readings: heart rate, SpO₂, temperature and blood pressure, shifted toward abnormal values for emergency cases.
- **operations.csv** (~13,000 rows) — hourly snapshots per department: bed occupancy, patients waiting and average wait time.

Each file was uploaded to its own sub-folder under the Bronze zone so that the Glue crawler would catalogue them as three distinct tables. The total volume (~4 MB) sits comfortably within Free Tier limits.

4. Implementation Workflow

The pipeline was built in the following order, with screenshots captured at each stage for the Implementation Guide:

- **S3 data lake** — created one bucket with Bronze/Silver/Gold/athena-results prefixes; uploaded the three CSVs to Bronze.
- **IAM** — created GlueHealthcareRole with the AWSGlueServiceRole and AmazonS3FullAccess managed policies (noted as over-permissive for a POC; see Limitations).
- **Glue Catalog & Crawler** — created the healthcare_db database and a crawler over the Bronze prefix; running it discovered three tables with correctly typed columns.
- **Athena** — configured a query-results location and ran the analytical SQL against the catalogued tables.
- **Glue ETL job** — ran a PySpark job that cleans the Bronze CSVs, writes typed Parquet to Silver, and builds three curated tables in Gold.
- **Step Functions** — a workflow runs the crawler then the ETL job automatically as one serverless pipeline.
- **Kinesis Data Firehose** — a delivery stream ingests real-time records into the Bronze zone, demonstrating streaming alongside batch.
- **QuickSight** — signed up, granted the service access to Athena and the S3 bucket (including the required Athena-workgroup write permission), created datasets in direct-query mode, built four visuals and published a dashboard.

4.1 Representative Athena queries

Four queries drive the dashboard: average occupancy per department; average wait time per department; readmission rate by age group (using a CASE expression to band ages); and a daily occupancy time series (parsing the timestamp string to a date). Aggregating the 0/1 readmission flag with AVG yields the readmission rate directly.

5. Results & Insights

The published QuickSight dashboard presents four visuals. The headline findings:

Visual	Finding	Operational insight
Occupancy by dept.	Emergency $\approx$ highest (~79%), General lowest (~49%)	Emergency is the most utilised unit
Wait time by dept.	Emergency longest (~54 min), Pediatrics shortest (~19 min)	Emergency is the main bottleneck
Readmission by age	Rises with age; 65+ highest (~26%)	Elderly patients are the priority risk group
Daily occupancy trend	Stable ~62–66% with daily variation	Demand is steady but spikes warrant proactive staffing

Together these tell a coherent story: the Emergency department is simultaneously the fullest and the slowest, making it the clear target for resource reallocation, while readmission risk concentrates in older patients — the natural population for a future predictive model.

6. Challenges

- **Region consistency** — Glue, Athena and QuickSight must all run in the same region as the S3 bucket (us-east-1); mismatches make services unable to see each other.
- **QuickSight permissions** — the most time-consuming issue: queries failed with an access error until the S3 bucket was granted the **Write permission for Athena Workgroup**, which is required but easy to miss during sign-up.
- **Timestamp typing** — the crawler inferred the timestamp column as a string, so the trend line initially sorted incorrectly; a calculated field using `parseDate()` fixed the date axis.
- **Single-analysis assembly** — visuals built across separate analyses had to be consolidated into one analysis (calculated fields and datasets are scoped per analysis) before publishing a clean dashboard.

7. Limitations

- **Over-broad IAM** — AmazonS3FullAccess was used for convenience; production should scope access to the single project bucket with least-privilege policies.
- **Synthetic data** — patterns are designed-in; real clinical data would be noisier and require more cleansing and validation.

8. Future Improvements

- **Real-time ingestion** with Kinesis Data Firehose for live vitals streaming and near-real-time dashboards.
- **Orchestration** with Step Functions and CloudWatch to schedule and monitor crawler → ETL → query as one workflow.
- **AI extension** — train a readmission-prediction model (SageMaker or Athena/Redshift ML) on the patients data, focusing on the high-risk elderly cohort.
- **Governance** — least-privilege IAM, Lake Formation permissions, and partitioning the Gold zone by date/department for faster, cheaper queries.

8b. AI Extension — Readmission Prediction

To address the machine-learning objective, we trained a model that predicts whether a patient will be readmitted within 30 days, using the same patients data from the pipeline. We compared **Logistic Regression** and a **Random Forest**, encoding categorical fields (department, diagnosis, admission type, gender) and using age and length of stay as numeric inputs.

The Random Forest reached about **68% accuracy** on a held-out test set. The most important predictors were **age** and **length of stay**, consistent with the readmission pattern seen in the dashboard. Because the data is synthetic, scores are modest (ROC-AUC ~0.6); on real clinical data with richer features the same approach would perform better. In a production setup the model would train in SageMaker on the curated Gold-zone data, and its risk scores would be written back to S3 and surfaced on the QuickSight dashboard. The full code and results are in the accompanying AI notebook (AI_Readmission_Model.ipynb).

9. Conclusion

The project delivers a working, serverless analytics pipeline that ingests synthetic healthcare data into an S3 data lake, catalogues and transforms it with AWS Glue, queries it with Athena, and visualises the results in QuickSight — entirely within the Free Tier. The dashboard surfaces clear operational insights around occupancy, wait times and readmission risk, and the architecture provides a reusable foundation ready for real-time ingestion and machine-learning extensions.